

Evolutionary Optimization of Activation Function Distributions

A Genetic Algorithm Approach with Mini-Batch Online Evaluation

Table of Contents

- 1 Overview
- 2 Fitness Evaluation Strategy
- 3 Evolutionary Algorithm
- 4 Operators
- 5 Summary

Overview

Setting

Consider a neural network with one or more hidden layers.

Notation:

- $v = \{v_1, v_2, \dots, v_n\}$ - raw output of a hidden layer
- $f = \{f_1, f_2, \dots, f_n\}$ - activation functions applied per neuron
- $f_i \in \mathcal{F}$ for $i = 1, \dots, n$
- $\mathcal{F}$ - the **function pool** (set of candidate activation functions)
- $\{B_0, \dots, B_m\}$ - Set of mini batches; m partitions of a full data set (batch) B

Goal

Optimize the *distribution* of activation functions across hidden layer neurons.

Table of Contents

- 1 Overview
- 2 Fitness Evaluation Strategy
- 3 Evolutionary Algorithm
- 4 Operators
- 5 Summary

Fitness Evaluation

- 1 Start with a network having random weights W and random AFs.
- 2 To evaluate a candidate AF vector:
 - Keep W fixed.
 - Measure error on the **current mini-batch B without any training**.
 - All candidates in the same generation are compared on **the same B** .
- 3 Selection, crossover, and mutation proceed as usual.
- 4 If the best candidate beats the current champion, **train the network for one mini-batch step** using B .
- 5 **After each generation:** advance to the next mini-batch.
After the end of mini-batch: shuffle the training set so batches change.

Key Advantage

Every candidate in the same generation is scored on the same B_λ , so fitness differences reflect the AFs, while avoiding overfitting (batch differ among generations).

Table of Contents

- 1 Overview
- 2 Fitness Evaluation Strategy
- 3 Evolutionary Algorithm**
- 4 Operators
- 5 Summary

Algorithm Overview – Step by Step

Procedure

- **Initialize:** population P of M individuals (random AF vectors), random weights W , mini-batch index $b \leftarrow 0$.
- **Set champion C :** best individual after no-training evaluation on the first mini-batch B_0 .
- **For each generation:**
 - Evaluate *all* individuals on the **same** current batch B_b (no training, W fixed) $\rightarrow$ fair fitness comparison.
 - Apply selection, crossover, mutation $\rightarrow$ new population P .
 - If best individual improves over C : replace C and **train W for one step** on B_b .
 - Advance batch index $b \leftarrow b + 1$; if end of mini batch reached, **shuffle** training set and reset $b \leftarrow 0$.
- **Return** champion C when stopping criterion is met.

Multi-layer Extension

Single Hidden Layer (Default)

Each individual is a vector of activation functions of length n (number of neurons).

$N > 1$ Hidden Layers

Two possible representations:

- **Concatenation:** stack the AF vectors of all layers into one individual
- **Alternative encoding:** to be determined per experiment

Table of Contents

- 1 Overview
- 2 Fitness Evaluation Strategy
- 3 Evolutionary Algorithm
- 4 Operators**
- 5 Summary

Selection

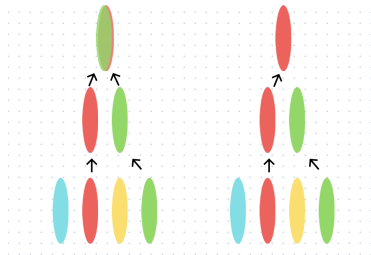
Strategy: Elitism + Tournament Selection (initial $k = 2$)

Algorithm 1 Selection(k)

```
1: Keep champion  $C$  in  $P^{t+1}$  (elitism)
2: while  $P^{t+1}$  is not full do
3:   Sample  $k$  individuals  $\rightarrow$  pick best as  $f_A$ 
4:   Sample  $k$  individuals  $\rightarrow$  pick best as  $f_B$ 
5:   Execute Crossover( $f_A, f_B$ ) with probability  $p_c$ 
6:   if Crossover executed then
7:     Add child  $f_C$  to  $P^{t+1}$ 
8:   else
9:     Add  $\arg \max_{\{f_A, f_B\}} \text{fitness}(f)$  to  $P^{t+1}$ 
10:  end if
11: end while
```

- Tournament size: $k = 2$
- Elitism preserves the champion

Tournament Selection In a Nutshell



Example with $k = 2$

- Select two individual pairs from P^t
- Pick the best one from each pair (parents)
- If Crossover: Make new individual with crossover from parents
- Else: Pick the best one from parents
- Add new individual in P^{t+1}

Crossover

Strategy: Uniform Crossover — two parents produce one child

Algorithm 2 Crossover(f_A, f_B)

- 1: Initialize f_C
 - 2: **for** each (f_{Ai}, f_{Bi}) in f_A, f_B **do**
 - 3: $f_{Ci} = f_{Ai}$ with probability 0.5
 - 4: Otherwise, $f_{Ci} = f_{Bi}$
 - 5: **end for**
 - 6: **return** f_C
-

Intuition: Flip a coin for each chromosome to decide whether or not it will be included in the off-spring

Parameters

- Initial crossover probability: $p_c = 0.8$
- Will remain **high** across experiments

Mutation

Strategy: Random replacement from the function pool

Algorithm 3 Mutation(f, l)

- 1: Select l elements from f uniformly at random
 - 2: **for** each selected element f_i **do**
 - 3: Replace f_i with a randomly drawn $f_s \in \mathcal{F} \setminus \{f_i\}$
 - 4: **end for**
-

Parameters

- Initial mutation probability: $p_m = 0.01$ (kept very low)
- Number of mutated elements: $l \approx 20\%$ of individual length

Table of Contents

- 1 Overview
- 2 Fitness Evaluation Strategy
- 3 Evolutionary Algorithm
- 4 Operators
- 5 Summary

Summary of Parameters

Parameter	Initial Value	Notes
Population size M	—	To be set per experiment
Max generations g	—	To be set per experiment
Mini-batch size	32	As proposed in the message
Tournament size k	2	4 individuals compared per slot
Crossover prob. p_c	0.8	Remains high
Mutation prob. p_m	0.01	Remains very low
Mutated elements l	$\approx 20\%$ of length	Per individual

Key Design Choices

- Fitness is evaluated **without training** using the current mini-batch.
- Weights are trained **only when the champion improves**.
- Elitism preserves the champion at all times.
- Uniform crossover provides gene scalability.